

BODHI: Precise OS Kernel Specification Inference

Zhiming Chang

Department of Applied Mathematics and Statistics
Johns Hopkins University
Baltimore, US
zchang9@jh.edu

Ziyang Li

Department of Computer Science
Johns Hopkins University
Baltimore, US
ziyang@cs.jhu.edu

Abstract—The formal verification of operating system kernels requires precise specifications that capture the intended behavior of system calls. Writing these specifications manually demands deep domain expertise, motivating the use of large language models (LLMs) to automate the process. However, in OSV-Bench, a benchmark of 245 specification generation tasks derived from the Hyperkernel OS kernel, the best reported Pass@1 is 55.10% [1]. We propose a domain knowledge prompting method (BODHI), which augments the standard few-shot prompt with a structured C-to-Python translation guide covering 15 categories of domain-specific translation patterns. Inspired by Structured Chain-of-Thought (SCoT) prompting [2], the guide organizes translation by separation of concerns, addressing pre-condition extraction and post-condition generation as distinct categories. Evaluated on nine models from six providers (Anthropic, Mistral, Amazon, DeepSeek, Meta, Alibaba), covering dense, mixture-of-experts and reasoning architectures, BODHI improves every model tested, with gains ranging from +11% to +32%. The best configuration (Claude Opus 4.6 + BODHI) reaches 96.73% Pass@1. BODHI reduces both syntax and semantic errors, with the strongest effect on models that have sufficient instruction-following capability to utilize structured reference material. These results demonstrate that domain knowledge injection is a model-agnostic technique that substantially bridges the gap between general-purpose code generation and formal specification synthesis.

Index Terms—Specification Generation, OS Kernel Verification, Large Language Models, Knowledge-Enhanced Prompting

I. INTRODUCTION

Formal software verification uses rigorous mathematical reasoning to prove that a system is free of bugs, which is crucial for safety-critical systems, such as operating system kernels, where software errors can have disastrous consequences [3]. However, constructing formal specifications, a prerequisite for verification, remains a bottleneck that requires deep domain expertise. For example, updating the seL4 microkernel to support efficient RPC modified less than 5% of the code, yet the resulting specification and proof adjustments took approximately one person-year of effort [4].

Large language models have shown strong capabilities in code generation [5], [6], but their performance in generating formal specifications remains limited. OSV-Bench [1], currently the largest benchmark for OS kernel specification generation, evaluated 12 large language models on 245 tasks derived from the Hyperkernel operating system [7]. Each task requires converting a C system call implementation into a

Python/Z3 state-machine specification. The best performing model achieved only 55.10% Pass@1, exposing a significant gap between natural language code generation and formal specification synthesis.

This gap results from several challenges inherent in the specification generation process:

- **Long-context complexity:** Each task prompt contains approximately 26K tokens, including a programming model, a small number of examples, a functional description and code for the target system call.
- **Semantic gap:** C kernel code must be converted to the declarative Z3 [8] Python specification, and these two programming styles are fundamentally different.
- **Interleaved concerns:** In C code, pre-conditions (error-checking logic) and post-conditions (state-changing logic) are structurally mixed, but they must be clearly distinguished in the specification.
- **Domain-specific knowledge:** Kernel API patterns such as page table entry (PTE) formulas, reference count semantics, TLB flush operations and IPC state-machines are highly specialized and not covered by general-purpose LLM training.

We adopt a simple but effective approach to address these challenges: we provide a comprehensive *structured C-to-Python translation guide*, building upon a standard set of prompts with a small number of examples. We call this approach BODHI. The guide is a 519-line reference document divided into 15 categories, offering clear translation patterns for every major construct encountered in the Hyperkernel domain. Drawing on the principles of Structured Chain-of-Thought (SCoT) prompting [2], it organizes the translation task into distinct concern categories (pre-condition extraction, post-condition patterns, field access syntax, and domain-specific formulas) rather than requiring the model to infer all patterns from a few examples alone.

In this paper, we make the following contributions:

- A domain-specific translation guide for OSV-Bench specification generation, covering 15 categories of C-to-Python/Z3 translation patterns derived from analysis of verified ground truth specifications.
- Comprehensive evaluation of BODHI on nine models from six providers with isolated API calls per-task, demonstrating model-agnostic effectiveness across di-

verse architectures.

- Detailed analysis showing that BODHI reduces both syntax and semantic errors, with the largest gains on mid-tier models that have sufficient instruction-following ability but incomplete domain knowledge.
- New state-of-the-art on the OSV-Bench at 96.73% Pass@1, substantially exceeding the previous best result of 55.10%.

II. BACKGROUND

A. OSV-Bench

The OSV-Bench [1] is a benchmark for evaluating LLMs in specification generation for OS kernel verification. It is built on Hyperkernel [7], a formally verified operating system kernel that uses the Z3 SMT solver [8] for verification.

Each of the 245 tasks requires generating a Python/Z3 state-machine specification for a system call. The specification defines pre-conditions and post-conditions. The task prompt includes: (1) a system verification assumption document, (2) a programming model defining available classes, constants, and functions, (3) few-shot examples of system call specifications, and (4) the target system call’s functional description and (potentially buggy) C implementation.

The 245 tasks cover 49 system calls with varying bug injections. Each task’s C implementation may contain one or more of five buggy types: incorrect pointer operations, incorrect privilege checks, memory leaks, buffer overflows, and missing bounds checks. In addition, 49 tasks contain correct implementations. A specification is correct if and only if the Z3 verifier produces results consistent with the oracle specification across all kernel implementation variants.

Specifications are verified in a Docker environment containing the Hyperkernel build system and Z3 solver [1], following the verification pipeline described in Section II-B. The primary metric is Pass@1: the fraction of tasks in which the LLM-generated specification passes verification on the first attempt.

The published OSV-Bench evaluation covers 12 LLMs, with the best Pass@1 reaching 55.10% (Doubao-1.5-pro) [1]. Notably, reasoning-enhanced models (o1, DeepSeek-R1) do not outperform standard instruction-following models, suggesting that specification generation requires domain-specific knowledge rather than general reasoning capability.

B. Z3 and the Verification Pipeline

The specifications in OSV-Bench are written using the Python API of Z3 [8], a Satisfiability Modulo Theories (SMT) solver. SMT solvers determine the satisfiability of logical formulas over theories such as fixed-width bitvectors and arrays, which directly model hardware registers, memory pages and kernel data structures in operating system kernels.

Each specification is a Python function that encodes the intended behavior of one system call as a state-machine transition. Formally, a specification for system call s is a function

$$f_s(\mathbf{S}, \mathbf{a}) \rightarrow (\phi, \mathbf{S}') \quad (1)$$

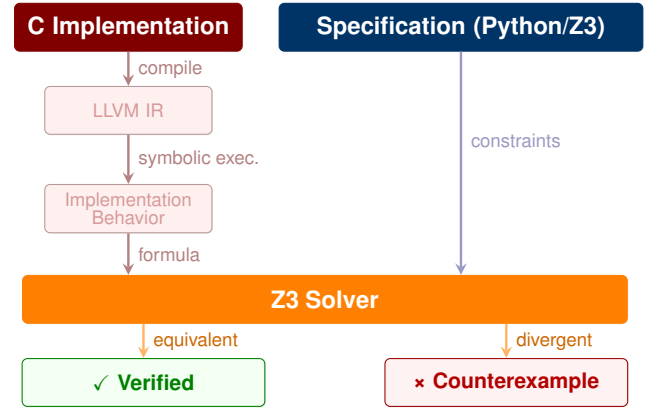


Fig. 1: Hyperkernel verification pipeline. The C implementation (left) is compiled to LLVM IR and symbolically executed to extract a behavioral formula. The specification (right) provides Z3 constraints. Z3 checks consistency: if the formula satisfies the constraints for all inputs, the system call is verified; otherwise, a counterexample is produced.

where $\mathbf{S}$ denotes the current kernel state (a collection of maps and scalar fields), $\mathbf{a}$ denotes the system call arguments, ϕ is the *pre-condition* (a Z3 Boolean expression representing the conjunction of all validity checks), and $\mathbf{S}'$ is the *post-state*. The post-state satisfies $\mathbf{S}' = \text{update}(\mathbf{S})$ when ϕ holds, and $\mathbf{S}' = \mathbf{S}$ otherwise.

Fig. 1 illustrates the verification pipeline used by Hyperkernel. The C implementation is compiled to LLVM intermediate representation (IR) [9], and the toolchain performs symbolic execution on the IR, extracting a logical formula that captures the implementation’s behavior over all possible inputs. Denoting the implementation’s extracted behavior as $g_s(\mathbf{S}, \mathbf{a})$, Z3 checks behavioral equivalence:

$$\text{verify}(s) \iff \forall \mathbf{a} : g_s(\mathbf{S}, \mathbf{a}) \equiv f_s(\mathbf{S}, \mathbf{a}) \quad (2)$$

If a discrepancy exists, Z3 produces a *counterexample*: a concrete $\mathbf{a}^*$ such that $g_s(\mathbf{S}, \mathbf{a}^*) \not\equiv f_s(\mathbf{S}, \mathbf{a}^*)$. If no such $\mathbf{a}^*$ exists, the system call is verified correct.

This *push-button* approach requires no manual proofs: given a correct specification, verification is fully automated. The bottleneck therefore shifts from proof construction to specification writing, precisely the task that OSV-Bench evaluates and that BODHI addresses.

C. Structured Chain-of-Thought (SCoT)

Chain-of-Thought (CoT) prompting [10] improves LLM reasoning by eliciting intermediate reasoning steps before the final answer. Structured Chain-of-Thought (SCoT) [2] generalizes the Chain-of-Thought framework to code generation, where sequential, branching, and looping program constructs are utilized to direct the intermediate reasoning procedure. Rather than generating free-form reasoning text, SCoT first produces a pseudocode skeleton that captures the control flow of the target program, then fills in the implementation details within that skeleton. This structural decomposition of the

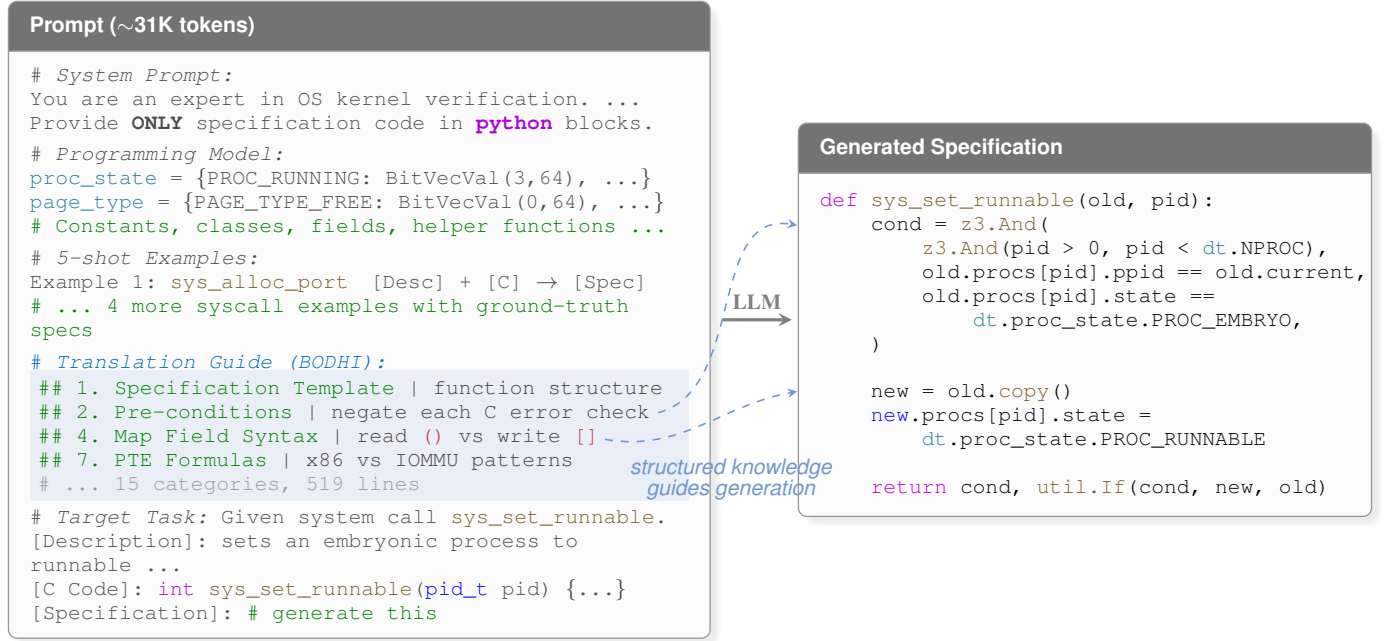


Fig. 2: The BODHI prompting pipeline. The prompt (left) contains five components; the *translation guide* (highlighted, our contribution) provides 15 categories of structured domain knowledge between the few-shot examples and the target task. Dashed arrows show how structured guide categories directly shape the generated specification (right): Category 2 guides pre-condition translation (red), Category 4 guides field access syntax (blue).

output generation process has shown improvements over basic CoT on code generation benchmarks such as HumanEval and MBPP [2]. We draw inspiration from SCoT but adapt it to the specification generation setting in a fundamentally different way. SCoT decomposes the *output* by structuring how the model generates code, while BODHI decomposes the *input* by organizing domain-specific translation patterns into structured reference material (Section III).

III. METHODOLOGY

Our method, BODHI, augments the standard few-shot prompt with a structured *C-to-Python Translation Guide*. This section describes the prompt structure, the guide’s design and construction, and the key principle of separation of concerns.

A. Prompt Structure

Fig. 2 illustrates the prompt composition for the baseline and BODHI configurations. Both configurations share four prompt components inherited from the OSV-Bench evaluation protocol [1]: (1) a *system prompt* (approximately 2K tokens) that instructs the LLM to generate Python/Z3 specifications; (2) the *programming model* (approximately 10K tokens), which defines the kernel state $\mathbf{S}$ (Eq. 1) including map declarations, bitvector widths and accessor conventions; (3) five expert-selected *few-shot examples* (approximately 12K tokens), each pairing a C system call with its oracle Z3 specification; and (4) the *target task* (approximately 2K tokens), comprising the natural-language functional description and the C implementation to be translated. BODHI adds a fifth

component: the *translation guide* (approximately 5K tokens), inserted between the few-shot examples and the target task.

The placement of the guide is deliberate. By positioning it after the few-shot examples and immediately before the target task, the guide occupies the most recent context when the model begins generation. This leverages the recency effect in transformer attention [11]: tokens closer to the generation boundary receive higher attention weight, ensuring that domain-specific patterns are maximally available during specification synthesis. The few-shot examples, placed earlier, establish the general input-output format, while the guide refines the model’s knowledge of domain-specific translation rules just before the model applies them.

The prompt is submitted as a single API call with no multi-turn interaction. The LLM generates the complete specification in one pass and the output is directly submitted to the Z3 verifier. This design keeps the method simple and reproducible: the only variable between the baseline and BODHI is the presence of the translation guide.

B. Translation Guide Design

The translation guide is a 519-line reference document covering 15 categories of domain-specific translation patterns (TABLE I).

It was constructed through an iterative process: (1) run baseline prompting on a subset of tasks, (2) classify failures into error types, (3) identify recurring patterns in the ground truth oracle specifications that address each error type, and (4) codify these patterns into structured guide categories. Each

TABLE I: Translation guide categories. Each category targets a specific class of specification errors.

ID	Category	Content	Error class	Addresses
1	Specification template	Standard function structure, special cases	Structural	Missing return format
2	Pre-condition translation	C error checks $\rightarrow$ Z3 negated conditions	Semantic	Incorrect guard logic
3	Post-condition patterns	<code>util.If</code> usage, <code>new.*</code> conventions	Syntax	Wrong conditional API
4	Map field syntax	<code>Read()</code> vs. <code>write[]</code> dual syntax	Syntax	Most common syntax error
5	Operator rules	Z3 Python API operator overloading	Syntax	Z3 API misuse
6	Constant prefixes	All constants use <code>dt.*</code> namespace	Syntax	Namespace violations
7	Page table PTE formulas	x86 vs. IOMMU, 7 mapping variants	Semantic	Core domain knowledge
8	Shadow metadata	5 shadow fields for page table mappings	Semantic	Implicit state omission
9	Reference counting	Ownership transfer refcnt updates	Semantic	Hidden in C, explicit in spec
10	TLB flush	x86 vs. IOMMU flush call differences	Completeness	Omitted operations
11	State pointers	<code>6_ptr_to_int</code> field exact names	Syntax	Naming errors
12	Field name mapping	C variable $\rightarrow$ Z3 path correspondence	Semantic	Translation accuracy
13	C helper functions	Helpers requiring inline expansion	Semantic	Abstraction mismatch
14	Available helpers	9 predefined helper functions	Syntax	API boundary
15	IPC system calls	<code>send/call/reply_wait/extintr</code> patterns	Completeness	Most complex syscall type

category targets a specific class of specification errors observed in preliminary experiments.

We used Claude Haiku 4.5 as the development model to maximize failure coverage during guide construction. The Claude Haiku 4.5 baseline achieved 50.20% Pass@1 (123/245), producing 49 syntax errors and 73 semantic errors across 122 failed tasks spanning 30 of the 49 system calls. This broad failure surface, nearly double the 16 failed system calls observed with Claude Opus 4.6, ensured that the guide would address error patterns common to models of varying capability levels. By designing the guide against a mid-tier model’s failures, the resulting categories are inclusive: patterns that a weaker model needs are a superset of those that a stronger model lacks, enabling the guide to generalize upward to more capable models as confirmed by the evaluation in Section IV-B. Manual inspection of the 122 failures revealed strong clustering around three subsystems (IPC, IOMMU and resource reclamation), motivating the three-tier organization of the guide.

The 15 categories were organized into three tiers based on the error analysis. The first tier targets *syntax-level* patterns that ensure ϕ and S' (Eq. 1) can be correctly constructed: specification template, pre-condition translation, post-condition patterns, map field syntax, operator rules and constant prefixes. These address the 49 syntax errors observed in the Haiku baseline, such as using parentheses instead of brackets for map field writes and omitting the `dt.` namespace prefix for constants. The second tier targets *domain-specific* semantic patterns: PTE formulas, shadow metadata, reference counting, TLB flush, state pointer names and field name mapping. These address the semantic errors concentrated in IOMMU and resource reclamation system calls, where the model consistently lacked specialized translation knowledge. The third tier targets *completeness* in the most complex system calls: C helper function expansion, available helper function inventory and IPC state machine patterns. These address the IPC and interrupt remapping failures, where even correct syntax and domain knowledge are insufficient without explicit multi-step coordination patterns. The resulting guide comprises 519 lines.

The 15 categories cover all recurring error patterns identified in the development set; the remaining failures are unique to individual system calls with no extractable general pattern. Importantly, the guide contains only general translation rules and illustrative patterns derived from the oracle specifications. It includes no task-specific code or complete solutions for any system call, functioning as a domain textbook rather than an answer key.

Fig. 3 shows an excerpt from two representative guide categories. The pre-condition category teaches the LLM that each C error check must be negated when translated to a Z3 guard condition. The map field syntax category addresses the most common syntax error observed in baseline experiments: Z3’s dual-access convention, where reading uses parentheses, but writing uses brackets.

C. Separation of Concerns

A key design principle of the guide is the *separation of concerns*: patterns for the pre-condition ϕ (error check negation) and the post-state S' (state updates) are organized into distinct categories (Eq. 1). This reflects a structural property of kernel system calls, where error-checking logic and state-mutation logic are naturally decoupled in the C source code.

Fig. 4 illustrates this with a real Hyperkernel system call (`sys_set_runnable`). The C implementation interleaves pre-conditions and post-conditions in a single control flow, but the Z3 specification separates them into two distinct regions: a `z3.And()` block for ϕ and a sequential assignment block for S' . The guide teaches the LLM to perform this structural transformation by providing separate pattern references for each region.

This separation is not merely organizational. In baseline experiments without the guide, the most common semantic errors involved the LLM incorrectly mixing pre-condition and post-condition logic. For example, applying negation to a state update expression or omitting an error check because it appeared syntactically similar to a state mutation. By structuring the guide around this separation, BODHI provides the LLM

Category 2: Pre-condition Translation	
<i>// Each C error return → negation in cond</i>	
C source	Z3 guard
<code>if (!is_pid_valid(pid)) return -E;</code>	<code>→ is_pid_valid(pid)</code>
<code>if (state != PROC_RUNNING) return -E;</code>	<code>→ old.procs[pid].state == dt.proc_state. PROC_RUNNING</code>
<code>if (n >= NPAGE) return -E;</code>	<code>→ z3.ULT(n, dt.NPAGE)</code>
Category 4: Map Field Syntax	
<i>// Read with () vs. Write with []</i>	
Read access	Write access
<code>old.pages[pn].data(idx)</code>	<code>new.pages[pn].data[idx] = val</code>
<code>old.procs[pid].ofile(fd)</code>	<code>new.procs[pid].ofile[fd] = val</code>
<code>old.procs[pid].nr_pages()</code>	<code>new.procs[pid].nr_pages[pn] += 1</code>

Fig. 3: Excerpts from two translation guide categories. *Category 2*: each C error check is negated to form the Z3 pre-condition guard. *Category 4*: Z3 map fields use parentheses for reads but brackets for writes, the most common syntax error in baseline experiments.

with distinct mental models for each translation task, reducing such cross-contamination errors.

IV. EVALUATION

We design our evaluation to answer four research questions: Does the translation guide improve specification generation accuracy across diverse models and providers? What types of error does the guide reduce? How does the guide’s contribution compare to model capability improvements? Which system call categories benefit the most from the guide?

A. Evaluation Setup

Baseline: Our baseline follows the OSV-Bench evaluation protocol [1]: a system prompt instructing the LLM to generate Python/Z3 specifications (2K tokens), followed by the definition of the programming model (10K tokens), five expert-selected few-shot examples (12K tokens), and the functional description of the target task and the implementation of C (2K tokens). The LLM generates the complete specification in a single call.

Models: We evaluate nine models from six providers: Claude Opus 4.6, Claude Sonnet 4, and Claude Haiku 4.5 (Anthropic); Devstral 2 123B (Mistral); Nova Premier (Amazon); DeepSeek-V3.2 and DeepSeek-R1 (DeepSeek); Llama 4 Maverick 17B-A3B (Meta); and Qwen3 80B-A3B (Alibaba).

This selection spans dense architectures, mixture-of-expert models and a reasoning-optimized model. The evaluations use greedy decoding (temperature 0.0) and `max_tokens` 4096, following the OSV-Bench prompt structure and verification pipeline [1]. Unlike the original evaluation, which averages three runs, we report single-run results; with greedy decoding, repeated runs are deterministic for most providers. DeepSeek-R1, as a reasoning model, does not support temperature configuration.

Task isolation: Each of the 245 tasks is processed as an independent API call with a fresh request body. No conversation history, system state or prior outputs are shared between tasks. This isolation is critical because multiple tasks share the same system call (with different bug injections) and leakage could inflate performance.

Token consumption: Each task prompt comprises approximately 26K input tokens for the baseline and approximately 31K for the BODHI version. The output specifications average approximately 340 tokens. The total token consumption in all 18 experiment configurations (9 models × 2 methods × 245 tasks) is approximately 126M tokens.

Verification: The generated specifications are verified using the OSV-Bench Docker environment, which contains the Hyperkernel build system and Z3 solver (v4.5.0) [1]. The verifier performs symbolic execution on compiled LLVM IR and checks behavioral equivalence (Eq. 2) with oracle specifications across all kernel implementation variants.

Evaluation Metrics: We adopt Pass@1 as the primary metric: each task is submitted once and the specification either passes or fails Z3 verification (Eq. 2). We also classify failures into *syntax errors* (the specification raises a Python or Z3 type error before verification completes) and *semantic errors* (the specification executes but produces verification results inconsistent with the oracle). This three-way split (pass / syntax / semantic) enables fine-grained analysis of how BODHI affects different error modes.

B. Main Results

Fig. 5 and TABLE II present our main results in nine models along with the published OSV-Bench baselines.

BODHI improves Pass@1 on every model tested, with gains ranging from +11.0 (Qwen3) to +31.8 (Devstral 2) percentage points, demonstrating that structured domain knowledge injection is effective regardless of the model provider, architecture or capability level.

Improvements are not monotonically related to baseline capability. Devstral 2 achieves the largest gain (+31.8%), reaching 80.0% with BODHI and surpassing all baselines except Claude Opus 4.6. Claude Haiku 4.5 and Claude Opus 4.6 follow. The three weakest baselines (DeepSeek-R1, Qwen3, Llama 4 Maverick) show moderate but consistent improvements of +11-16%. This suggests that the effectiveness of BODHI depends on a model’s ability to utilize structured reference material: models with sufficient instruction-following capability benefit substantially, while models limited by out-

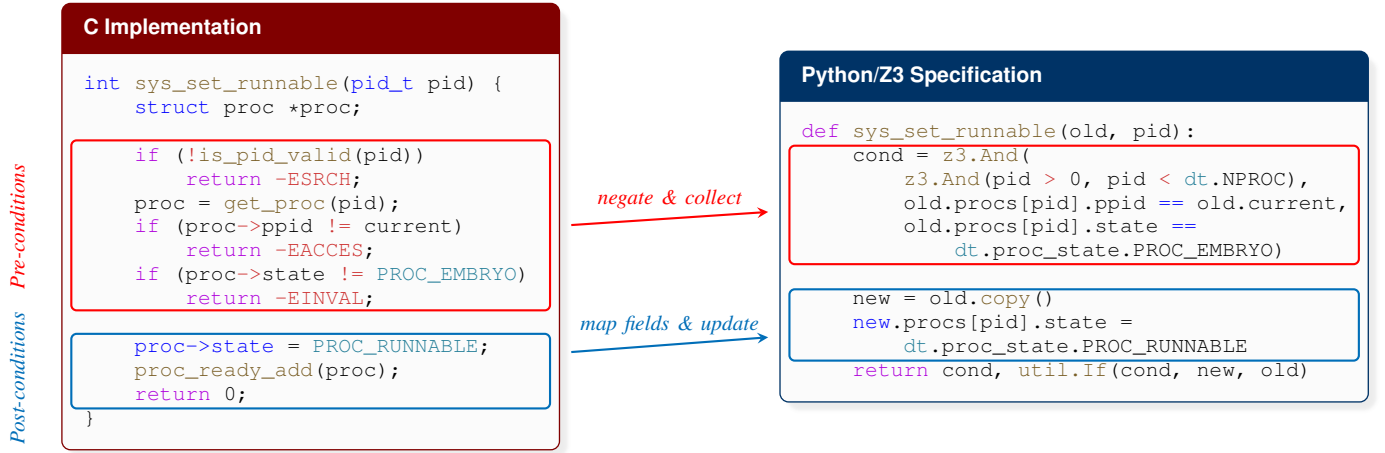


Fig. 4: Separation of concerns in the C-to-Z3 translation (`sys_set_runnable`). Red overlay: pre-conditions (C error checks are negated and collected into a `z3.And()` guard). Blue overlay: post-conditions (C state mutations are mapped to Z3 field assignments).

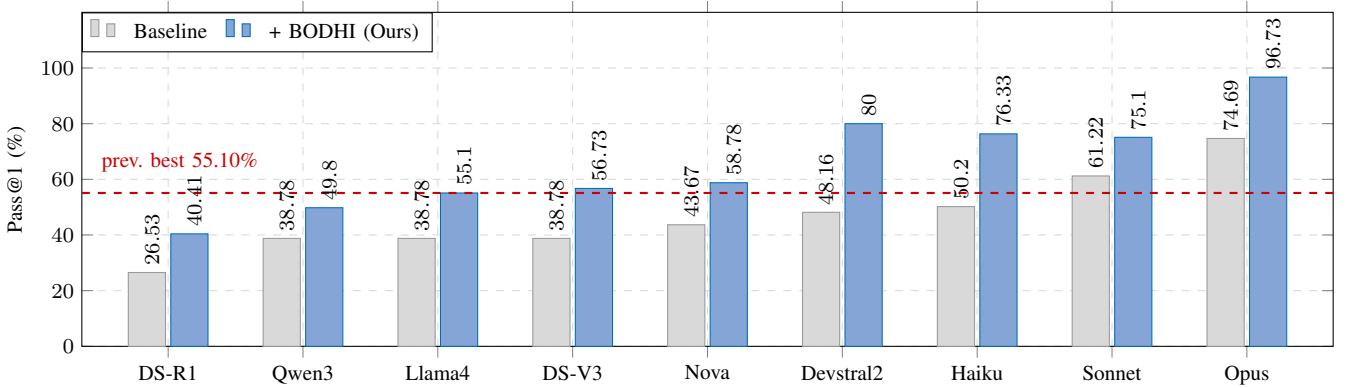


Fig. 5: Pass@1 on OSV-Bench across nine models from six providers, sorted by baseline performance. Each pair compares baseline (gray) with BODHI (blue). Dashed red line: previous state-of-the-art (Doubao-1.5-pro [1]). BODHI improves every model with significant gains.

put format compliance (DeepSeek-R1) or fundamental code generation capability derive smaller but still positive gains.

Six models with BODHI surpass the previous state-of-the-art (Doubao-1.5-pro, 55.10% [1]): Claude Opus 4.6, Devstral 2, Claude Haiku 4.5, Claude Sonnet 4, Nova Premier and DeepSeek-V3.2; Llama 4 Maverick matches it. The best configuration reaches 96.73%, failing on only 8 of 245 tasks. The guide improves performance across all bug categories for all models, with the greatest gains appearing in the buffer overflow and memory leak categories, where domain-specific PTE formulas and reference counting patterns are the most relevant.

C. Error Analysis

Fig. 6 breaks down the 245 tasks into three categories for each model configuration: pass, syntax error (specification does not execute due to Z3 type mismatches, undefined functions, or Python errors) and semantic error (specification executes but verification fails due to incorrect logic). We select three representative models spanning weak, mid-tier,

and strong baselines. The inner ring shows the three-way split (pass, syntax, semantic); the outer ring further divides syntax errors into *type/sort mismatches* (Z3 bit-vector size or type conflicts) and *API/reference errors* (undefined Z3 functions or attributes), and semantic errors into *domain pattern errors* (failures in syscalls requiring specialized patterns such as IPC, PTE formulas, or IOMMU mappings) and *translation logic errors* (incorrect condition negation or field mapping in structurally simpler syscalls).

Strong models (Claude Opus 4.6, Devstral 2, Claude Sonnet 4, Claude Haiku 4.5): BODHI substantially reduces both error types, as shown in panels (b) and (c). Devstral 2’s syntax errors drop from 31.0% to 8.2%; the outer ring decomposes this reduction into two components: type/sort mismatches are completely eliminated (6.1% → 0%), while API/reference errors decrease from 24.9% to 8.2%. For Claude Opus 4.6, the baseline semantic errors are almost exclusively domain pattern errors, with translation logic errors nearly absent (0.4%), indicating that Claude Opus 4.6 already handles

TABLE II: Pass@1 (%) on OSV-Bench. Published baselines from [1]; our results use isolated per-task API calls with temperature 0.0. All nine models show consistent improvement with BODHI.

Model	Incorr. Ptr.	Incorr. Priv.	Mem. Leak	Buf. Ovfl.	Bounds Chk.	Correct	Total
<i>Published baselines [1]</i>							
Doubao-1.5-pro	50.70	48.21	45.95	40.74	52.78	63.27	55.10
DeepSeek-Chat	38.02	39.29	36.49	44.44	43.52	51.02	46.53
Claude Sonnet 3.5	39.44	41.96	39.19	48.15	39.81	46.94	44.90
<i>Our baselines (without BODHI)</i>							
Claude Opus 4.6	70.42	72.32	60.81	62.96	69.44	79.59	74.69
Claude Sonnet 4	54.93	51.79	52.70	46.30	55.56	75.51	61.22
Claude Haiku 4.5	39.44	41.96	41.89	42.59	43.52	59.18	50.20
Devstral 2	46.48	41.96	47.30	42.59	48.15	51.02	48.16
Nova Premier	39.44	35.71	37.84	27.78	40.74	48.98	43.67
DeepSeek-V3.2	30.99	36.61	31.08	29.63	34.26	38.78	38.78
Llama 4 Maverick	33.80	33.04	25.68	35.19	40.74	42.86	38.78
Qwen3	43.66	29.46	32.43	20.37	33.33	44.90	38.78
DeepSeek-R1 [†]	21.13	22.32	18.92	29.63	28.70	24.49	26.53
<i>Our results (with BODHI)</i>							
Claude Opus 4.6	97.18 (+26.8)	95.54 (+23.2)	94.59 (+33.8)	98.15 (+35.2)	96.30 (+26.9)	97.96 (+18.4)	96.73 (+22.0)
Claude Sonnet 4	77.46 (+22.5)	66.96 (+15.2)	75.68 (+23.0)	79.63 (+33.3)	70.37 (+14.8)	85.71 (+10.2)	75.10 (+13.9)
Claude Haiku 4.5	76.06 (+36.6)	66.07 (+24.1)	70.27 (+28.4)	75.93 (+33.3)	72.22 (+28.7)	83.67 (+24.5)	76.33 (+26.1)
Devstral 2	74.65 (+28.2)	72.32 (+30.4)	75.68 (+28.4)	85.19 (+42.6)	77.78 (+29.6)	85.71 (+34.7)	80.00 (+31.8)
Nova Premier	56.34 (+16.9)	53.57 (+17.9)	52.70 (+14.9)	37.04 (+9.3)	57.41 (+16.7)	63.27 (+14.3)	58.78 (+15.1)
DeepSeek-V3.2	47.89 (+16.9)	53.57 (+17.0)	51.35 (+20.3)	59.26 (+29.6)	52.78 (+18.5)	63.27 (+24.5)	56.73 (+18.0)
Llama 4 Maverick	52.11 (+18.3)	48.21 (+15.2)	39.19 (+13.5)	46.30 (+11.1)	57.41 (+16.7)	59.18 (+16.3)	55.10 (+16.3)
Qwen3	54.93 (+11.3)	37.50 (+8.0)	44.59 (+12.2)	38.89 (+18.5)	47.22 (+13.9)	55.10 (+10.2)	49.80 (+11.0)
DeepSeek-R1 [†]	35.21 (+14.1)	33.93 (+11.6)	27.03 (+8.1)	44.44 (+14.8)	37.04 (+8.3)	53.06 (+28.6)	40.41 (+13.9)

[†]Reasoning model; temperature not configurable. High syntax error rate (59%/53%) due to output format non-compliance.

Our baseline (26.53%) is lower than the published result (40.00% [1]), likely due to API version differences and single-run evaluation.

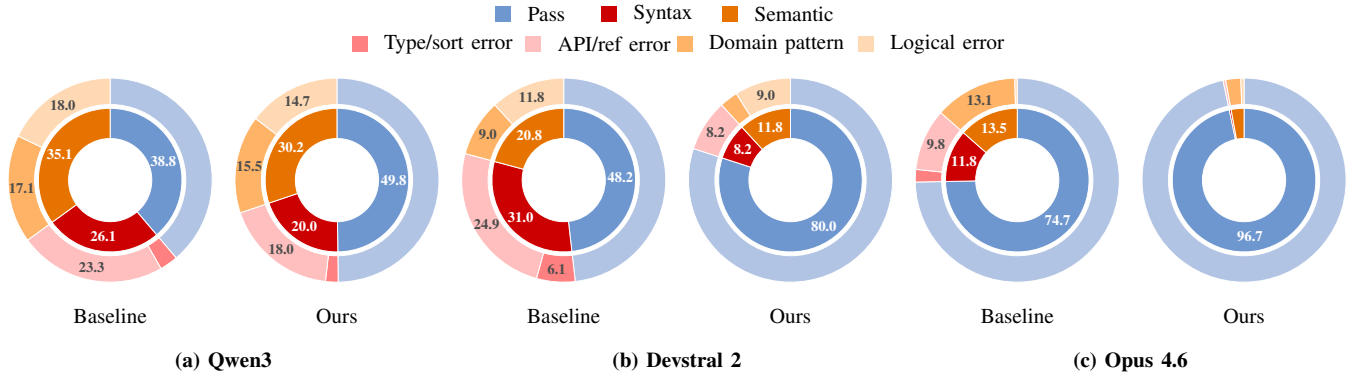


Fig. 6: Error distribution as nested donut charts for three representative models (weak/mid/strong). Inner ring: pass/syntax/semantic split (percentages shown). Outer ring: syntax errors split into type/sort mismatches and API/reference errors; semantic errors split into domain pattern errors and translation logic errors. BODHI eliminates type/sort errors and reduces API errors across all tiers. Opus baseline failures are almost exclusively domain pattern errors; BODHI resolves most of them.

simple translations well. BODHI reduces domain pattern errors to 2.4%, leaving only 8 failures.

Weaker models (Nova Premier, DeepSeek-V3.2, Llama 4 Maverick, Qwen3): BODHI reduces both error types, but substantial residual errors remain, as illustrated by Qwen3 in panels (a). Unlike strong models, Qwen3’s semantic errors are split nearly equally between domain pattern and translation logic, indicating that weaker models struggle with both specialized patterns and basic translation. BODHI provides moderate reductions in both sub-categories, but 30.2% semantic errors persist.

DeepSeek-R1 represents a special case: as a reasoning-optimized model, it exhibits the highest syntax error rate (59.2% baseline, 53.1% with BODHI). Manual inspection reveals that DeepSeek-R1 frequently fails to produce output in the required ```python``` format, instead generating reasoning in natural language. Its low semantic error rate suggests that when DeepSeek-R1 does produce valid specifications, they tend to be semantically reasonable.

Residual failure analysis: We examine the 8 remaining Claude Opus 4.6 + BODHI failures. The failures concentrate on two system calls: `sys_reclaim_intremap` fails in

TABLE III: Selected system calls showing BODHI impact (Claude Opus 4.6, Pass@1 across 5 variants per syscall).

System Call	Category	Baseline	+ BODHI	Guide sections used
call_proc	IPC	0/5	5/5	IPC patterns
send_proc	IPC	0/5	5/5	IPC patterns
reply_wait_proc	IPC	0/5	5/5	IPC patterns
sys_alloc_iommu_pt	IOMMU alloc	0/5	5/5	PTE formulas, shadow metadata
sys_alloc_iommu_pdpt	IOMMU alloc	1/5	5/5	PTE formulas, field mapping
sys_reclaim_page	Page reclaim	2/5	5/5	Refcnt, state pointers
sys_reclaim_iommu_frame	IOMMU reclaim	2/5	5/5	PTE formulas, refcnt
sys_alloc_intremap	Interrupt	0/5	4/5	Field mapping, helpers
sys_lseek	File management	5/5	4/5	(regression)
sys_map_pml4	Page mapping	5/5	5/5	(already correct)

all 5 variants (*semantic errors*): this system call handles the reclamation of the interrupt mapping table, coordinating interactions among the IOMMU state, the interrupt vectors and the ownership of the device. Neither baseline nor BODHI produces a correct specification, suggesting that the required pattern (coordinating multiple hardware subsystem states during resource reclamation) exceeds the guide’s coverage. `sys_lseek` fails in 1 variant (*syntax error*): the model generates `z3.SLT` for a signed comparison, but Z3 v4.5.0 does not expose this function. The remaining 2 failures occur in `sys_alloc_intremap` and `sys_alloc_io_bitmap`. (*semantic errors*) The concentration of most failures in the interrupt remapping subsystem suggests a clear direction for guide improvement.

Z3 verifier feedback: When a specification fails, the Z3 verification pipeline produces diagnostically distinct feedback depending on the error class. For *syntax errors*, the Python/Z3 runtime raises an exception (e.g. `TypeError`, `Z3Exception`) with a message that pinpoints the fault. Messages such as `Sort mismatch: expected BitVec(64), got BitVec(32)` directly identify the offending field and the expected type. This feedback is precise and machine-readable, making it suitable for automated repair: an LLM could receive the traceback and re-generate only the erroneous expression. For *semantic errors*, the specification executes successfully but Z3 finds a concrete $\mathbf{a}^*$ violating Eq. 2 and produces a *counterexample*: an assignment of bitvector values to all symbolic inputs, such as `pid = 0×3` and `ppid = 0×1`, under which the implementation and specification diverge. While counterexamples formally prove incorrectness, interpreting them requires mapping raw bitvector values back to domain-level semantics (process states, page table entries), a non-trivial task that limits their direct utility for automated repair. This asymmetry, actionable feedback for syntax errors but opaque feedback for semantic errors, suggests that a verification-feedback repair loop (Section VII) would be most effective for syntax failures, where the error signal is both precise and self-contained.

D. Model vs. Method Contribution

With nine models spanning six providers, we can now robustly disentangle the model capability from the BODHI contribution. The mean improvement with BODHI is +18.7%,

with all nine models showing statistically significant gains in a controlled comparison (same model, same tasks, identical evaluation protocol).

Three observations emerge:

- The contribution of BODHI is *model-agnostic*: it improves models regardless of provider (Anthropic, Mistral, Amazon, DeepSeek, Meta, Alibaba), architecture (dense, MoE) or capability level. This rules out the hypothesis that the effectiveness of guide is specific to a particular model family’s training data.
- The effect of BODHI is *comparable to or greater than a full model generation upgrade*: For example, the improvement from the Claude Sonnet 3.5 baseline to the Claude Sonnet 4 baseline is +16.3%, while applying BODHI to Sonnet 4 yields an additional +13.9%. For Devstral 2, BODHI alone (+31.8%) exceeds the effect of any single model upgrade observed.
- The relationship between baseline capability and BODHI gain is *non-linear*: The greatest gains are accrued to mid-tier models (Mistral Devstral 2: +31.8%, Claude Haiku 4.5: +26.1%) rather than the strongest or weakest. We hypothesize that BODHI requires sufficient instruction-following ability to parse and apply the guide’s patterns, but yields the largest marginal gains when the model’s intrinsic domain knowledge is incomplete, a condition more characteristic of mid-tier models.

E. Per-Syscall Analysis

To understand where the guide helps the most, we analyze the 49 system calls (each appearing in 5 task variants) in OSV-Bench [1]. We take Claude Opus 4.6 as an example: TABLE III shows representative system calls with notable improvement patterns.

The greatest improvements occur in system calls that require domain-specific translation patterns. All three IPC system calls improve from 0/5 to 5/5, as the guide provides dedicated IPC state-machine patterns that are difficult to infer from the few-shot examples alone. Similarly, IOMMU-related system calls benefit substantially from the PTE formula and shadow metadata categories. System calls with straightforward logic show no improvement, as the baseline already succeeds. A minor regression occurs for `sys_lseek`, caused by a mismatch

in the Z3 API version, as discussed in the residual failure analysis.

F. Qualitative analysis

To illustrate how BODHI changes model behavior concretely, we examine three representative cases where the baseline (Claude Opus 4.6) fails but BODHI succeeds.

Case 1: IPC state machine logic: The `call_proc` system call requires checking whether the receiver process is willing to accept messages from the current sender:

✗ Baseline: disjunctive check

```
z3.Or(
old.procs[pid].ipc_from == z3.BitVecVal(0,
dt.pid_t),
old.procs[pid].ipc_from == old.current), # wrong
semantics
```

✓ BODHI: implication check

```
z3.Implies(old.procs[pid].ipc_from != 0,
old.procs[pid].ipc_from == old.current),
```

The baseline conflates two independent conditions into a disjunction, which allows for invalid states. The IPC pattern section of BODHI specifies the correct implication semantics: if the receiver has a designated sender (`ipc_from != 0`), it must match the current process.

Case 2: IOMMU page table entry formula: For `sys_alloc_iommu_pt`, the baseline applies the x86 PTE formula to an IOMMU entry:

✗ Baseline: x86-style PFN shift

```
pfn = z3.UDiv(old.pages_ptr_to_int,
z3.BitVecVal(dt.PAGE_SIZE, 64)) + to
new.pages[frm].data[index] = (pfn <<
dt.DMAR_PTE_ADDR_SHIFT) | perm # wrong formula
```

✓ BODHI: IOMMU byte-addressed

```
new.pages[frm].data[index] =
(new.pages_ptr_to_int + to * dt.PAGE_SIZE) | perm
```

The x86 page tables encode entries as shifted page-frame numbers, but IOMMU entries use direct byte addresses. Without the explicit distinction of the guide between these two formula families (Category 7 in Table I), the model defaults to the more common x86 pattern.

Case 3: Map field read/write syntax: A common syntax error involves Z3’s dual-access convention for map fields:

✗ Baseline: parentheses for write

```
new.pages[pn].data(index) = value # TypeError
```

✓ BODHI: brackets for write

```
new.pages[pn].data[index] = value
```

In the Z3 Python API, the map fields use parentheses `()` for reads, but brackets `[]` for writes. This dual convention is not

documented in the programming model document provided in the prompt (Section II-A) in OSV-Bench [1], making it a frequent source of syntax errors. The Category 4 of BODHI (Table I) explicitly lists all dual-access fields with both access patterns.

These cases illustrate the three error classes addressed by BODHI: semantic logic errors (Case 1), domain knowledge gaps (Case 2) and syntax convention errors (Case 3).

V. RELATED WORK

LLM-based generation of formal artifacts: Code2Inv [12] pioneered neural approaches to formal reasoning with reinforcement learning for loop invariant inference. Baldur [13] applied LLMs to whole-proof generation in Isabelle/HOL. SpecGen [14] explored LLM-based specification generation for general-purpose programs. More recently, LLMs have been applied to generate other formal artifacts: IRIS [15] combines LLMs with static analysis to synthesize security vulnerability detectors, and QLCoder [16] uses LLMs to generate CodeQL queries for static analysis. These works share a common theme with BODHI: LLMs generating domain-specific formal code (Z3 specifications, CodeQL queries, Isabelle proofs) where general code generation capability is insufficient and structured domain knowledge is required.

Neurosymbolic programming: Neurosymbolic method [17], [18] concerns combining neural modules along with symbolic algorithms on downstream applications. Existing frameworks like [19]–[21] introduce programming languages that integrate neural computation with symbolic reasoning, enabling neural networks to leverage logical rules during inference. BODHI operates at a different level of abstraction: rather than embedding symbolic reasoning into the model architecture, we inject domain knowledge into the prompt, allowing unmodified LLMs to produce formally verifiable artifacts. Both approaches address the fundamental challenge of bridging neural and symbolic reasoning, but BODHI requires no model modification or training.

Prompting methods for code generation: Chain-of-Thought prompting [10] and its structured variant SCoT [2] improve code generation by generating intermediate reasoning. BODHI adapts SCoT principles to specification generation by providing structured domain knowledge rather than eliciting reasoning chains: the structure is in the reference material, not the output.

OS kernel verification: Hyperkernel [7] introduced push-button verification for an OS kernel using Z3. SeL4 [4] demonstrated comprehensive formal verification of a microkernel. OSV-Bench [1] established the first LLM benchmark for this domain. Our work builds on OSV-Bench by demonstrating that domain-specific prompting can substantially improve LLM performance on specification generation.

Knowledge-enhanced prompting: Retrieval-augmented generation (RAG) [22] and tool-augmented approaches inject external knowledge into LLM prompts. Chen et al. [23] study how API documentation aids LLM code generation, finding that example code in documentation contributes more

than descriptive text — a finding consistent with our guide design, which emphasizes concrete translation patterns over abstract rules. Our translation guide serves a similar purpose to RAG but is hand-crafted for the specific domain, providing higher precision than retrieved passages. The guide can be viewed as a form of structured few-shot documentation: more comprehensive than examples alone, but more targeted than a full manual.

VI. DISCUSSION

A. Threats to Validity

Internal validity: The translation guide was constructed using failure analysis on Claude Haiku 4.5, which is also one of the nine evaluated models. This raises the concern that improvements on Claude Haiku 4.5 may partly reflect overfitting to the development model’s failure modes rather than general translation knowledge. We mitigate this in two ways: (1) the guide contains only general translation rules and illustrative patterns, not task-specific solutions; and (2) the largest improvement is observed on Devstral 2 (+31.8%), a model from a different provider that was not involved in guide development, demonstrating cross-model generalization. A second concern is potential oracle knowledge leakage: the guide patterns were derived from inspecting oracle specifications. However, the guide encodes only reusable domain rules (e.g. IOMMU PTEs use byte addressing, not 4K-page addressing) rather than complete specifications for any task. The 15 categories cover general patterns applicable across multiple system calls, functioning as a domain textbook rather than an answer key.

External validity: Our evaluation is conducted on a single benchmark (OSV-Bench) targeting a single codebase (Hyperkernel) [1]. The 245 tasks span 49 system calls within one x86 microkernel, and the translation guide is specific to the Hyperkernel programming model and Z3 specification language. Whether the guide-based prompting approach generalizes to other verification targets, specification languages, or OS kernels remains an open question. However, the methodology, systematic failure analysis followed by structured knowledge injection, is not inherently domain-specific and could be applied to other formal verification tasks.

Construct validity: We use Pass@1 as the primary metric, which is binary: a specification either passes or fails Z3 verification. This metric does not capture partial correctness. For example, a specification that correctly translates 95% of a system call’s logic but mishandles one edge case receives the same score as one with entirely incorrect logic. The three-way error classification (pass/syntax/semantic) partially addresses this limitation by distinguishing failure modes. Additionally, we report single-run results with greedy decoding rather than averaging multiple runs, which eliminates sampling variance for most models but means our results reflect a single deterministic configuration rather than expected performance across temperature settings.

B. Limitations

Guide construction cost: The translation guide required analyzing 49 oracle specifications to identify patterns. This one-time effort does not generalize to new verification domains without corresponding analysis.

Output format dependency: BODHI assumes that the model can produce syntactically valid Python in the requested format. Models that struggle with format compliance (e.g. DeepSeek-R1, which outputs reasoning traces rather than code blocks in 53-59% of tasks) derive limited benefit. A preprocessing or re-prompting step to enforce output format could improve results for such models.

Single-call vs. multi-stage: BODHI injects the guide into a single LLM call. A multi-stage approach that first analyzes the system call and then generates pre-conditions and post-conditions separately might yield further improvements, particularly for models with smaller context windows.

VII. CONCLUSION

We demonstrate that BODHI, a domain knowledge prompting method with structured translation guides, significantly improves LLM performance in formal specification generation across a broad range of models. Evaluated on nine models from six providers, BODHI improves every model tested, with a mean gain of +18.7 percentage points and the best configuration reaching 96.73% Pass@1, substantially exceeding the previous best of 55.10%. The key insight is that specification generation requires domain-specific translation knowledge (page table formulas, reference counting semantics, field naming conventions) that cannot be reliably inferred from few-shot examples alone but can be effectively communicated through a structured reference document.

The model-agnostic effectiveness of BODHI across diverse architectures (dense, mixture-of-experts, reasoning) and providers demonstrates that the technique is not tied to any particular model family. The largest gains accrue to mid-tier models with sufficient instruction-following capability but incomplete domain knowledge, pointing to a productive division of labor between human domain experts (who construct the guide) and LLMs (who apply it at scale).

Future work includes: (1) exploring multi-stage generation where pre-conditions and post-conditions are generated in separate LLM calls, (2) adding a verification-feedback repair stage to address remaining failures, and (3) extending to other verification domains to test transferability of the BODHI framework.

REFERENCES

- [1] S. Li, J. Jiang, T. Zhao, and J. Shen, “OSVBench: Benchmarking LLMs on specification generation tasks for operating system verification,” *arXiv preprint arXiv:2504.20964*, 2025.
- [2] J. Li, G. Li, Y. Li, and Z. Jin, “Structured chain-of-thought prompting for code generation,” *arXiv preprint arXiv:2305.06599*, 2023.
- [3] V. D’silva, D. Kroening, and G. Weissenbacher, “A survey of automated techniques for formal software verification,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 7, pp. 1165–1178, 2008.

- [4] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood, “seL4: Formal verification of an OS kernel,” in *Proc. ACM SIGOPS 22nd Symp. Operating Systems Principles*, 2009, pp. 207–220.
- [5] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton, “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, 2021.
- [6] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, “A survey on large language models for code generation,” *arXiv preprint arXiv:2406.00515*, 2024.
- [7] L. Nelson, H. Sigurbjarnarson, K. Zhang, D. Johnson, J. Bornholt, E. Torlak, and X. Wang, “Hyperkernel: Push-button verification of an OS kernel,” in *Proc. 26th Symp. Operating Systems Principles (SOSP)*, 2017, pp. 252–269.
- [8] L. D. Moura and N. Bjørner, “Z3: An efficient SMT solver,” in *Proc. Int. Conf. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008, pp. 337–340.
- [9] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *Proc. Int. Symp. Code Generation and Optimization (CGO)*, 2004, pp. 75–86.
- [10] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [11] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 157–173, 2024.
- [12] X. Si, H. Dai, M. Raghothaman, M. Naik, and L. Song, “Learning loop invariants for program verification,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [13] E. First, M. N. Rabe, T. Ringer, and Y. Brun, “Baldur: Whole-proof generation and repair with large language models,” in *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2023.
- [14] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, “SpecGen: Automated generation of formal program specifications via large language models,” *arXiv preprint arXiv:2401.08807*, 2024.
- [15] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.
- [16] C. Wang, Z. Li, S. Dutta, and M. Naik, “QLCoder: A query synthesizer for static analysis of security vulnerabilities,” *arXiv preprint arXiv:2511.08462*, 2025.
- [17] S. Chaudhuri, K. Ellis, O. Polozov, R. Singh, A. Solar-Lezama, and Y. Yue, “Neurosymbolic programming,” *Foundations and Trends in Programming Languages*, vol. 7, no. 3, pp. 158–243, 12 2021. [Online]. Available: <https://doi.org/10.1561/25000000049>
- [18] J. Zhu, C. Shen, Z. Li, J. Yu, Y. Chen, and K. Pei, “Locus: Agentic predicate synthesis for directed fuzzing,” *arXiv preprint arXiv:2508.21302*, 2025.
- [19] Z. Li, J. Huang, and M. Naik, “Scallop: A language for neurosymbolic programming,” *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, pp. 1463–1487, 2023.
- [20] P. Biberstein, Z. Li, J. Devietti, and M. Naik, “Lobster: A gpu-accelerated framework for neurosymbolic programming,” in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ACM, Dec. 2025, p. 147–162. [Online]. Available: <http://dx.doi.org/10.1145/3760250.3762232>
- [21] Z. Li, J. Huang, J. Liu, F. Zhu, E. Zhao, W. Dodds, N. Velingker, R. Alur, and M. Naik, “Relational programming with foundational models,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 9, p. 10635–10644, Mar. 2024. [Online]. Available: <http://dx.doi.org/10.1609/aaai.v38i9.28934>
- [22] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [23] J. Chen, S. Chen, J. Cao, J. Shen, and S.-C. Cheung, “When LLMs meet API documentation: Can retrieval augmentation aid code generation just as it helps developers?” *arXiv preprint arXiv:2503.15231*, 2025.